

Tipos de Dado Abstrato: Listas

Introdução

- Tradicionalmente conhecidos como Tipos de Dado Abstrato, são algumas Estruturas de Dados básicas e importantes para a construção de algoritmos mais bem elaborados;

Listas

LISTAS - INTRODUÇÃO

Listas

- Listas são conjuntos de elementos, objetos, variáveis, tarefas, ou qualquer coisa que se possa enumerar e formar um conjunto;
- As listas estão presentes em nossa vida, desde o nosso nascimento, por exemplo, com a lista de compras que nossos pais tiveram que fazer para nós.

Listas

- Exemplo de Lista de Compras:
 - 5Kg de farinha;
 - 2Kg de açúcar;
 - 500g de carne moída;
 - 2Kg de arroz;
 - 4L de leite;
 - 1Kg de feijão;
 - Etc..

Listas

- Exemplo de Lista Telefônica:
 - Asdf de Zxcv: (44) 4444-4444
 - Beutrano Cruz: (33) 3333-3333
 - Ciclano da Silva: (22) 2222-2222
 - Fulano de Tal: (11) 1111-1111

Listas, Filas e Pilhas

COMPORTAMENTO DE UMA LISTA

Comportamento de uma Lista

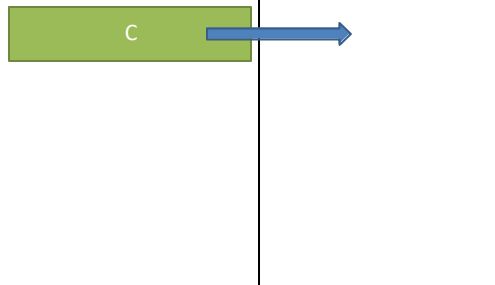
- Lista: Vazia!



Comportamento de uma Lista

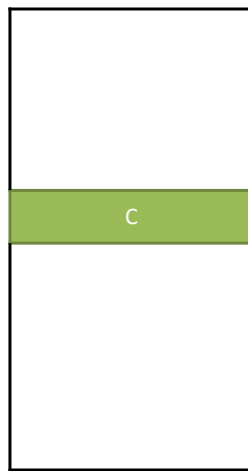
- Lista:

— Inserir “C”



Comportamento de uma Lista

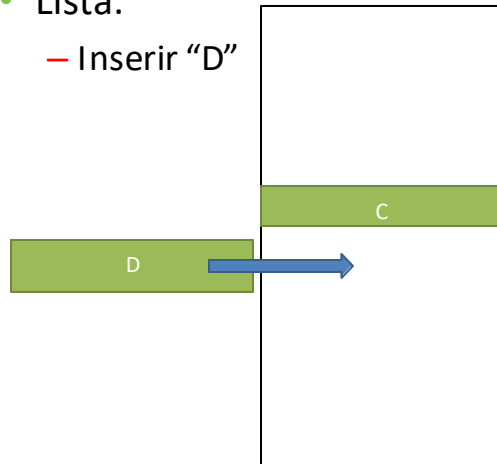
- Lista:



Comportamento de uma Lista

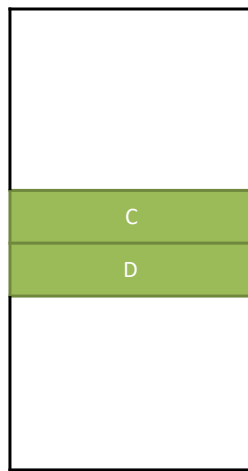
- Lista:

— Inserir “D”



Comportamento de uma Lista

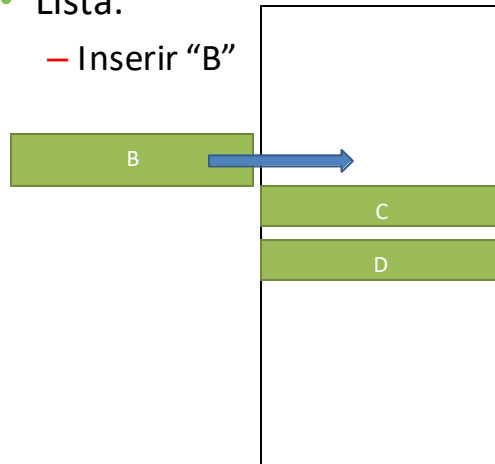
- Lista:



Comportamento de uma Lista

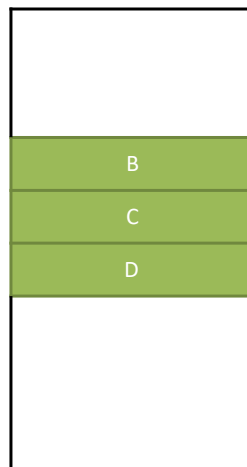
- Lista:

— Inserir “B”



Comportamento de uma Lista

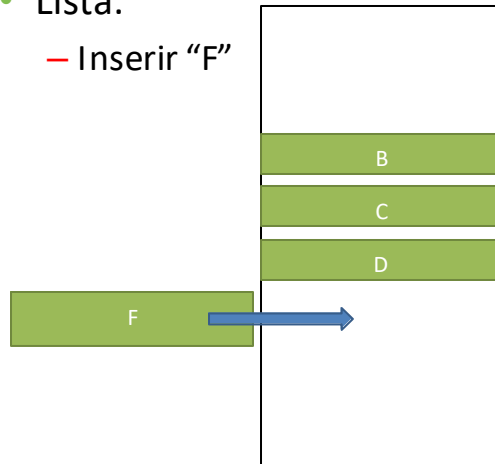
- Lista:



Comportamento de uma Lista

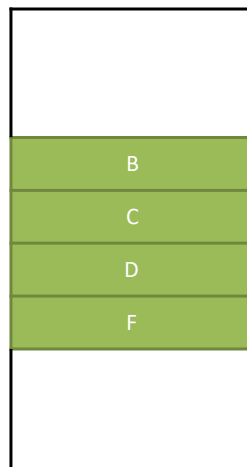
- Lista:

— Inserir “F”



Comportamento de uma Lista

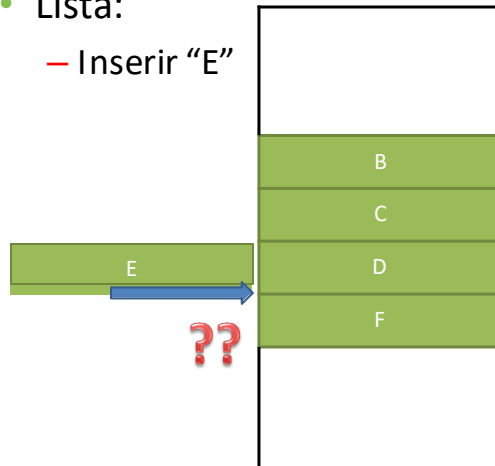
- Lista:



Comportamento de uma Lista

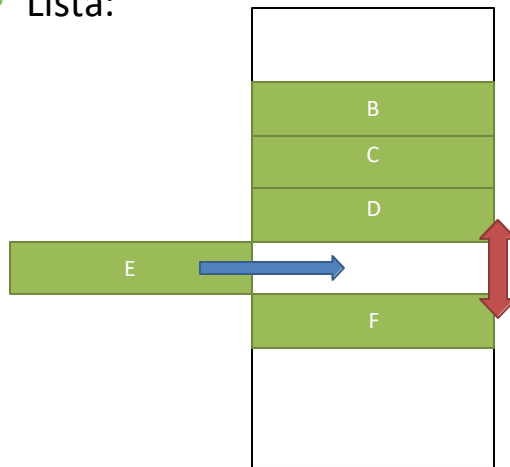
- Lista:

— Inserir “E”



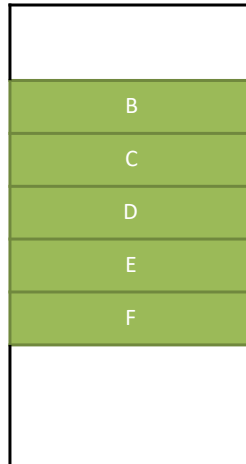
Comportamento de uma Lista

- Lista:



Comportamento de uma Lista

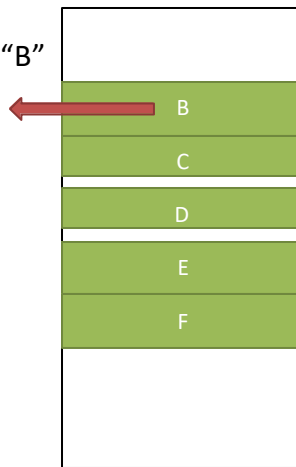
- Lista:



Comportamento de uma Lista

- Lista:

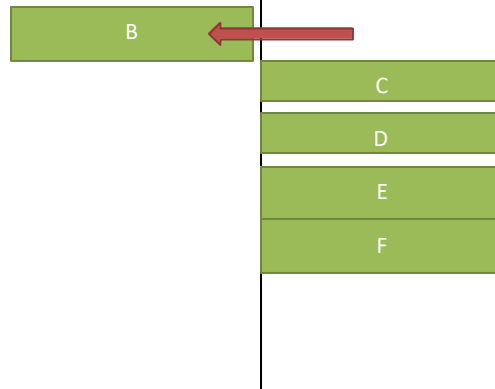
— Remover “B”



Comportamento de uma Lista

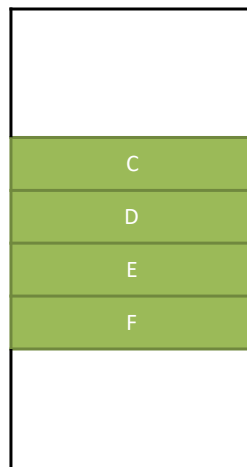
- Lista:

— Remover “B”



Comportamento de uma Lista

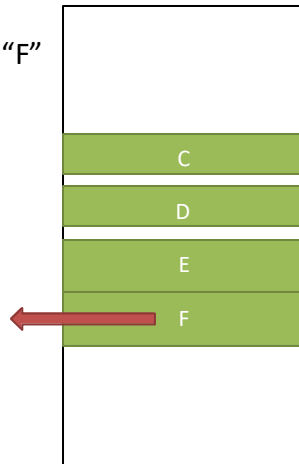
- Lista:



Comportamento de uma Lista

- Lista:

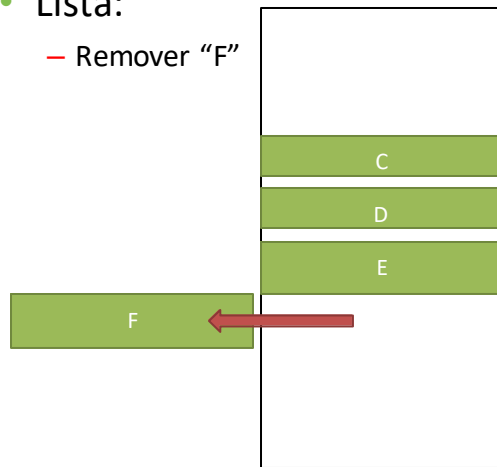
— Remover “F”



Comportamento de uma Lista

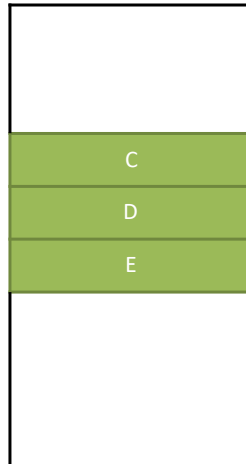
- Lista:

— Remover “F”



Comportamento de uma Lista

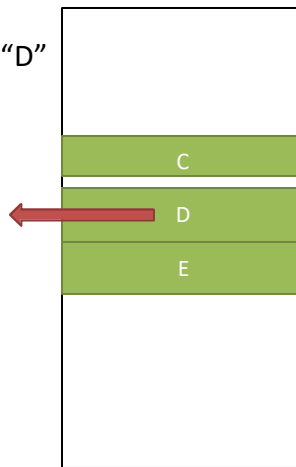
- Lista:



Comportamento de uma Lista

- Lista:

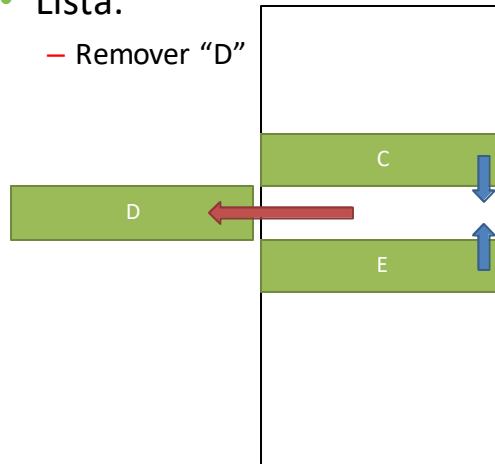
— Remover “D”



Comportamento de uma Lista

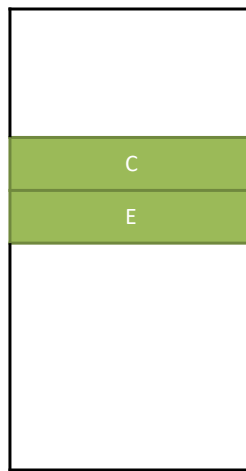
- Lista:

— Remover “D”



Comportamento de uma Lista

- Lista:



Listas, Filas e Pilhas

LISTAS - IMPLEMENTAÇÃO

Listas

- Implementando as listas:
 - As listas podem ser implementadas de várias formas, mas num aspecto mais geral podemos separar em duas principais:
 - Em **Arrays**; ou
 - **Encadeadas**.

Listas em Arrays

- **Em Arrays:**

- Imagine que a lista anterior tinha posições fixas e pré-determinadas:

- Um array é uma estrutura com posições fixas, cada elemento da lista deve ser colocado em uma posição no array;
 - Ao inserir ou excluir um elemento, talvez seja necessário realocar todos os demais elementos.

Listas em Arrays

- Prós:

- Criar um array de qualquer tamanho é muito simples;
 - Não há necessidade de compreender ponteiros ou referências;

- Contras:

- Limitações quanto ao tamanho de memória;
 - Custo computacional maior;
 - Alocação de memória exagerada.

Listas Encadeadas

- Encadeado, Dicionário Houaiss:
 - *adjetivo*
 - 1. disposto ou ligado por ou como por cadeias; ordenado, junto;
 - 2. preso, submetido;

Listas Encadeadas

- Prós:
 - Extremamente eficiente no custo de memória e de processamento;
 - Nunca acarreta em movimentar todos os elementos;
- Contras:
 - Envolve conceitos mais avançados de programação:
 - Ponteiros ou Referências.

Listas Encadeadas

- Para criarmos uma lista encadeada, precisamos primeiro definir o que será armazenado nela;
- Por exemplo, para criarmos uma lista de contatos, gostaríamos de armazenar os nomes, telefones e e-mails de diversas pessoas:

Listas Encadeadas

- Exemplo de elemento “Contato” da lista:



Listas Encadeadas

- Exemplo da Idéia de Encadeamento:



- Mas como fazer isto?

Listas Encadeadas

- Conforme vamos criando elementos na memória do computador, estes elementos vão espalhando e desconexos;
- Para criar listas encadeadas precisamos criar elementos que façam **referência** a outro elemento, ou seja, indiquem onde podemos encontrar um outro elemento.

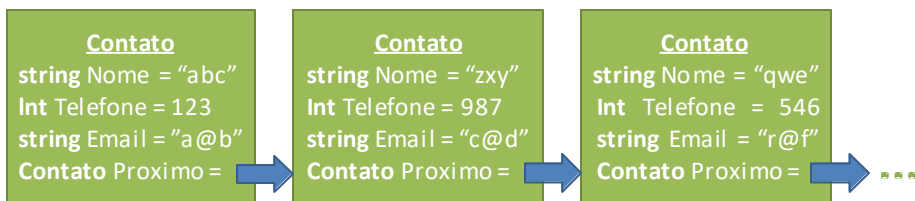
Listas Encadeadas

- Exemplo de elemento encadeado:



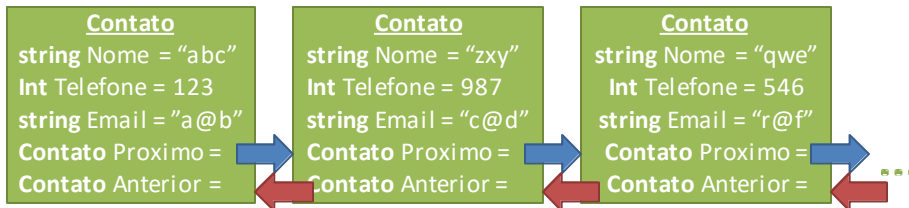
Listas Encadeadas

- Exemplo com Elemento **Encadeado**:



Listas Encadeadas

- Exemplo **Duplamente** Encadeado:



Listas Encadeadas

- Iniciando uma lista vazia:
 - **Contato** = [];
 - **list**;
 - O “*valor*” de referência **null** é usado para quando ainda **não existe um objeto** na memória para qual a variável irá fazer referência;
 - O último elemento da lista aponta para **null**.
- Iniciando uma lista com 1 elemento:
 - **Contato** = [#];

Listas Encadeadas

— Criando a Lista:

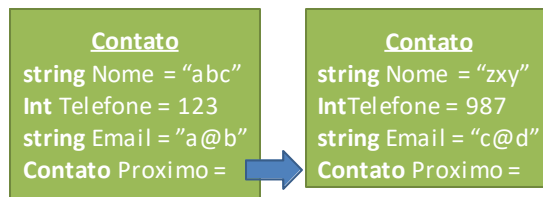
- **Contato** Inicio_Lista = [0];
- **Contato** Fim_Lista = [-1];
- Inicio_Lista.Nome = "abc";
- Inicio_Lista.Telefone = 123;
- Inicio_Lista.Email = "a@b";
- Inicio_Lista.Proximo = null;

```
Contato  
string Nome = "abc"  
Int Telefone = 123  
string Email = "a@b"  
Contato Proximo =
```

Listas Encadeadas

— Adicionando um segundo elemento:

- **Contato** novo = [];
- novo.Nome = "zxy";
- novo.Telefone = 987;
- novo.Email = "c@d";
- novo.Proximo = null;
- **Fim_Lista.Proximo = novo;**
- **Fim_Lista = novo;**



LISTA EM PYTHON

Listas

Durante o desenvolvimento de software, independentemente de plataforma e linguagem, é comum a necessidade de criar, manter e manipular conjuntos de dados. Tais conjuntos são muito variados, tanto quanto à natureza dos dados como com relação às quantidades envolvidas.

O tipo lista é a ferramenta disponível para atender a essa demanda e representa o mais genérico, versátil e poderoso tipo sequencial.

As listas podem ser empregadas para armazenar coleções de números inteiros ou reais, palavras, nomes ou quaisquer outras informações necessárias à solução de algum problema computacional.

Assim como o `string`, é um tipo sequencial composto por elementos organizados de modo linear, na qual cada um pode ser acessado a partir de um índice que representa sua posição na coleção, iniciando em zero. Em função disso, a lista suporta muitas das mesmas operações do tipo `string`.

Quanto ao conteúdo, os elementos de uma lista podem ser qualquer tipo de objeto. Além disso, as listas são mutáveis, de modo que seus elementos podem ser alterados livremente e a qualquer momento pelo programador.

Operações básicas com listas

É possível criar uma lista atribuindo-se um conjunto de dados entre colchetes [] a um identificador de objeto. Se não houver dados entre os colchetes, cria-se uma lista vazia.

O acesso individual aos elementos da lista é feito por meio de seu índice e cada elemento é mutável, podendo, portanto, ser alterado. Além disso, esses elementos podem ser de quaisquer tipos, compondo uma lista heterogênea.

Caso se queira excluir um elemento da lista, basta utilizar o comando `del`, passando como parâmetro o elemento a ser excluído.

```
L = []
type(L)
print(L)
L = [10, 15, 20, 25, 30]
print(L)
L[0]
len(L)
L[0] = 8
print(L)
A = [3, 7.5, 'txt']
print(A)
type(A)
type(A[0])
type(A[1])
type(A[2])
del(A[1])
A
```

```
L = []
type(L)
print(L)
L = [10, 15, 20, 25, 30]
print(L)
L[0]
len(L)
L[0] = 8
print(L)
A = [3, 7.5, 'txt']
print(A)
type(A)
type(A[0])
type(A[1])
type(A[2])
del(A[1])
A
```


Operações básicas com listas

Deve-se ter o cuidado de interpretar o resultado produzido pelo operador de adição “+”. Quando os operadores envolvidos forem elementos da lista, a operação será definida em função dos tipos desses elementos.

```
L = [10, 15, 20, 25, 30]
A = [3, 7.5, 'txt']
X = L[0] + A[1]
X
X = L + A
X
type (X)
```

```
L = [10, 15, 20, 25, 30]
A = [3, 7.5, 'txt']
X = L[0] + A[1]
X
X = L + A
X
type (X)
```

Fatiamento de listas

Utiliza-se a mesma notação `L[início:final]` para selecionar o sublista de `L` que começa na posição dada pelo indexador início e termina na posição dada pelo indexador final – 1. Também é possível utilizar a notação que inclui o passo: `L[início:final:passo]`, que, dentro do intervalo `[início:final]`, seleciona o primeiro elemento de cada subintervalo dado pelo valor contido no passo.

```
L = [1, 3, 5, 9, 11, 13, 15, 17, 19, 21, 23]
L [0:3]
L [4:10]
L [:5]
L [5:]
L [0:8:3]
L[::4]
```

```
L = [1, 3, 5, 9, 11, 13, 15, 17, 19, 21, 23]
L [0:3]
L [4:10]
L [:5]
L [5:]
L [0:8:3]
L[::4]
```

Multiplicador

Quando aplicado a listas, o operador multiplicativo “*” necessita de uma lista de origem e um número inteiro e produz uma nova lista com diversas repetições da lista original.

```
A = [3,7] * 3
A
L = [5] * 10
L
```

```
A = [3,7] * 3
A
L = [5] * 10
L
```

Conversão para lista

Operação que pode ser útil em muitos casos é a conversão de um string em uma lista.

```
S = "Um texto"      S = "5 7 8 8.8 12"
L = list (S)        L = S.split()
L                    L

                        S = "5;7;8.8;12"
                        L = S.split(";")
                        L
```

```
S = "Um texto"
L = list (S)
L

                        S = "5 7 8 8.8 12"
                        L = S.split()
                        L

                        S = "5;7;8.8;12"
                        L = S.split(";")
                        L
```

Operador in

O operador in permite ao programador verificar se um valor está presente em uma lista

```
L = [3, 6, 9]
9 in L
5 in L
5 not in L

Caes = ["Labrador", "Poodle", "Terrier"]
a = "Lassie"
if a in Caes:
    print ("Boa escolha")
else:
    print ("Não temos essa raça")
```

```
L = [3, 6, 9]
9 in L
5 in L
5 not in L
```

```
Caes = ["Labrador", "Poodle", "Terrier"]
a = "Lassie"
if a in Caes:
    print ("Boa escolha")
else:
    print ("Não temos essa raça")
```

Outros Métodos

```
L = [3, 6, 9]
L
L.append(2)
L
L.insert(2,15)
L
A = [22, 32, 42]
L.append(6)
L.extend(A)
L
L.reverse()
L.count(6)
L
L.index(9)
L.sort()
L.pop(3)
L
L.sort(reverse=True)
L.remove(6)
L
L.clear()
L = ["dado", "uva", "caixa", "lata", "casa"]
L.sort()
L
L = [23, 7.7, 3.9, 'txt', '3eml']
L.sort()
```

Listas Vinculadas

É possível utilizar o operador de atribuição “=” para atribuir uma lista existente a outro identificador.

```
L = [2, 4, 6, 8, 10]
V = L
V
V[0] = 15
V
L
id(L)
id(V)
C = L.copy()
id(C)
C[0] = 2
C
L
```

Listas Aninhadas

Listas dentro de uma lista

```
L = [[2, 4, 6], [1, 2, 3]]
L[0]
type (L[0])
L[1]
L[1][0]
L[1][2]
A = [7, 8, 9, 10]
L.append(A)
L
```

```
L = [[1, 2, ['3.1', '3.2', ['3.3.1', ['3.3.2.1', '3.3.2.2'], '3.3.3']]], [4,5,6]]
L[0]
L[0][2]
L[0][2][2]
L[0][2][2][2][1]
```

Exercício

Crie 2 listas: uma com 5 nomes(João, Maria, Kleber, Caio e Sarah) e outra com 5 valores em reais(R\$) correspondentes ao saldo da conta do usuário(2350.20; 540.50; 300.00; 830.15 e 150.00), e usando laços de repetição imprima os dados da seguinte forma:

Saída/Impressão:

LISTA DE CLIENTES - BANCO XXXXXXXXX

NOME	SALDO
------	-------

nome0	saldo0
-------	--------

nome1	saldo1
-------	--------

Listas, Filas e Pilhas

FILAS

Filas

- O que é uma fila em nosso cotidiano?
- As filas são diferentes das listas?
 - Em quais sentidos?
- Onde usamos filas em nosso cotidiano?
- Detalhe o funcionamento de uma fila.

Filas

- Existem muitos exemplos de fila no mundo real:
 - Uma fila de banco;
 - No ponto de ônibus;
 - Um grupo de carros aguardando sua vez no pedágio;
 - Entre outros.

Filas

- Uma fila é um conjunto de itens a partir do qual podem-se eliminar itens numa extremidade (chamada início da fila) e no qual podem-se inserir itens na outra extremidade (chamada final da fila).



Filas

- Filas são casos especiais de listas;
- **Obs:** Nas listas, quando precisávamos criar um novo elemento, poderíamos inseri-lo ou removê-lo de qualquer posição da lista, exemplos:
 - Na primeira posição;
 - Na última posição; ou
 - Em qualquer parte no meio da lista.

Filas

- Numa **fila** existe uma regra básica a ser seguida:
 - Primeiro a Chegar é o Primeiro a Sair;
 - Do inglês: FIFO – *First In, First Out*;
- Um novo elemento da **fila** somente pode ser **inserido** na última posição(fim da fila);
- Um elemento só pode ser **removido** da **primeira** posição (inicio da fila).

Filas

- Tem um sentido de chegada:
 - Fila vazia.



Filas

- Inserindo Elementos:
 - Inserir o elemento “G”



Filas

- Inserindo Elementos:
 - O elemento entra na última posição.



Filas

- Inserindo Elementos:
 - E avança até a primeira posição disponível.



Filas

- Inserindo Elementos:
 - Inserir o elemento "B"



Filas

- Inserindo Elementos:
 - O elemento entra na última posição



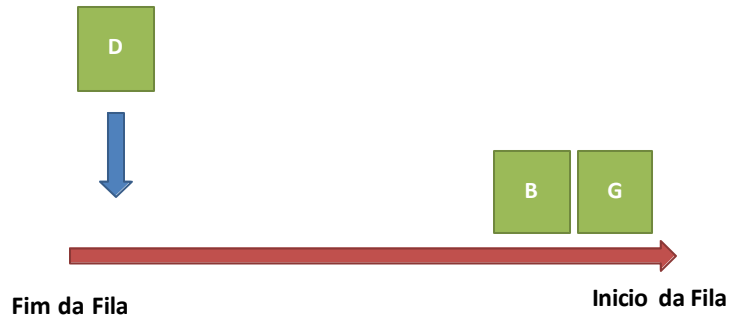
Filas

- Inserindo Elementos:
 - E avança até a primeira posição disponível.



Filas

- Inserindo Elementos:
 - Inserir o elemento “D”



Filas

- Inserindo Elementos:
 - O elemento entra na última posição



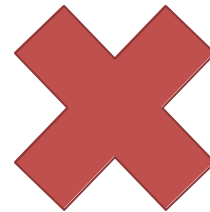
Filas

- Inserindo Elementos:
 - E avança até a primeira posição disponível.



Filas

- Removendo Elementos:
 - Remover o elemento B?
 - Não podemos remover elementos que não estejam no inicio da fila!
 - Da mesma forma, o elemento D não pode ser removido!

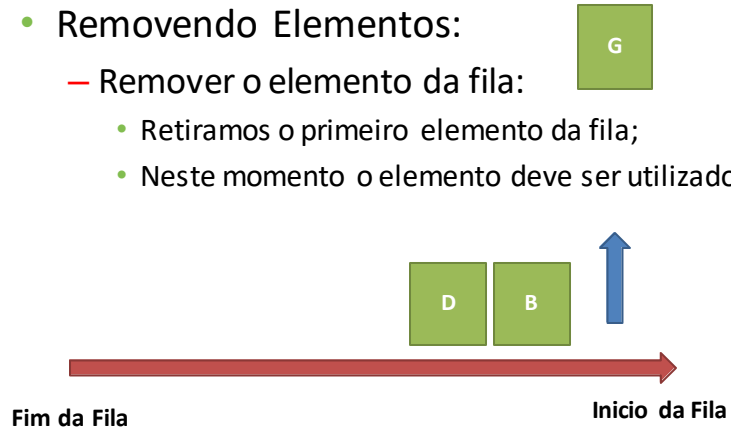


Filas

- Removendo Elementos:

- Remover o elemento da fila:

- Retiramos o primeiro elemento da fila;
 - Neste momento o elemento deve ser utilizado.

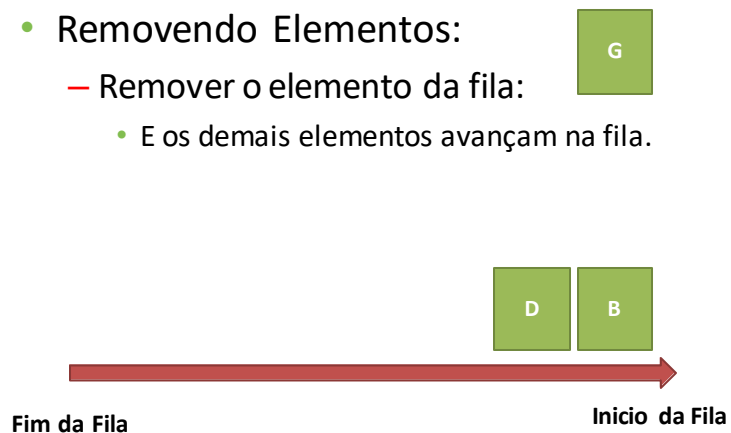


Filas

- Removendo Elementos:

- Remover o elemento da fila:

- E os demais elementos avançam na fila.



Filas

- Exemplos de uso de filas na computação:
 - Filas de impressão:
 - Impressoras tem uma fila, caso vários documentos sejam impressos, por um ou mais usuários, os primeiros documentos impressos serão de quem enviar primeiro;
 - Filas de processos:
 - Vários programas podem estar sendo executados pelo sistema operacional. O mesmo tem uma fila que indica a ordem de qual será executado primeiro;
 - Filas de tarefas:
 - Um programa pode ter um conjunto de dados para processar. Estes dados podem estar dispostos em uma fila, onde o que foi inserido primeiro, será atendido primeiro.

Filas

- Variações de Filas:
 - Fila de Prioridades:
 - Cada item tem uma prioridade. Elementos mais prioritários podem ser atendidos antes, mesmo não estando no início da fila;
 - Fila Circular:
 - Neste tipo de fila os elementos nem sempre são removidos ao serem atendidos, mas voltam ao fim da fila para serem atendidos novamente mais tarde.

IMPLEMENTANDO FILAS

Filas

- As filas podem ser implementadas em listas encadeadas ou em vetores;
- Vetores:
 - Devemos ter duas variáveis indicando a posição do início e do fim da fila;
- **Lista Encadeada:**
 - Devemos ter duas referências, uma ao elemento de início da fila e outra ao elemento do fim da fila.

Collections - Fila

- **Fila:** ([documentação](#)) ([IME-USP](#))
 - Construir:
 - `Queue()`
 - Adicionar:
 - `enqueue(item)`
 - Remover:
 - `dequeue()`
 - Testar:
 - `isEmpty()`
 - Tamanho:
 - `size()`

```
class Queue:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def enqueue(self, item):
        self.items.insert(0,item)

    def dequeue(self):
        return self.items.pop()

    def size(self):
        return len(self.items)

q=Queue()

q.enqueue(4)
q.enqueue('dog')
q.enqueue(True)
print(q.size())
```

Listas, Filas e Pilhas

PILHAS

Pilhas

- Um dos conceitos mais úteis na ciência da computação é o de pilha;



Pilhas

- Como eram as listas?
 - Insere, remove ou utiliza qualquer elemento inserido;
- Como eram as filas?
 - Insere apenas no fim da fila, utiliza e remove apenas o primeiro elemento inserido;

Pilhas

- Como são as Pilhas?
 - Insere-se elementos no topo da pilha;
 - Remove-se ou utiliza-se apenas o elemento que estiver no topo da pilha!
- LIFO (ou FILO):
 - **L**ast **I**n, **F**irst **O**ut;
 - Último a entrar, primeiro a sair;

Pilhas

- Pilha Vazia: **Topo** = **null**;

Pilha *p*

Pilhas

- Pilha Vazia: **Topo** = **null**;

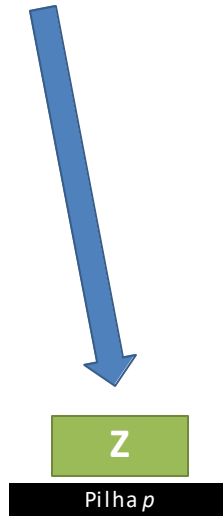
- Inserindo elemento **Z**

Z

Pilha *p*

Pilhas

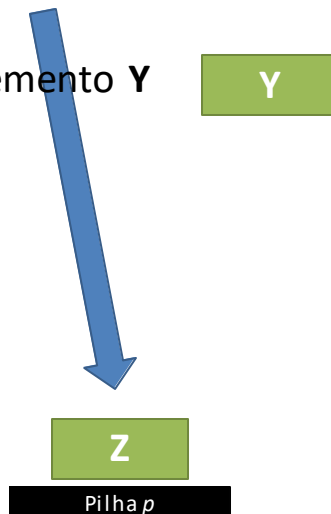
- Pilha Vazia: **Topo**



Pilhas

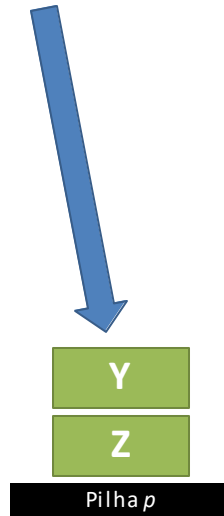
- Pilha Vazia: **Topo**

- Inserindo elemento **Y**



Pilhas

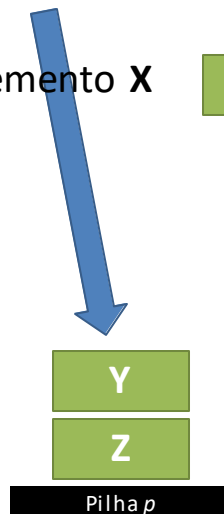
- Pilha Vazia: **Topo**



Pilhas

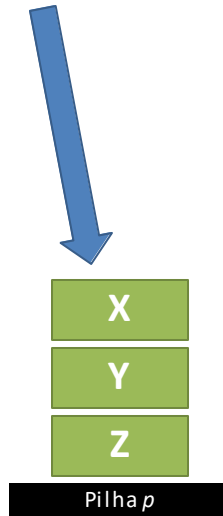
- Pilha Vazia: **Topo**

- Inserindo elemento **X**



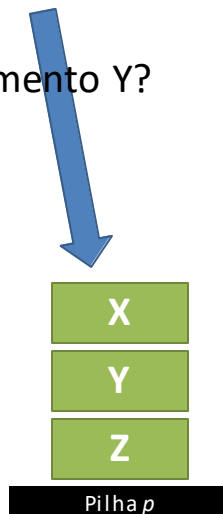
Pilhas

- Pilha Vazia: **Topo**



Pilhas

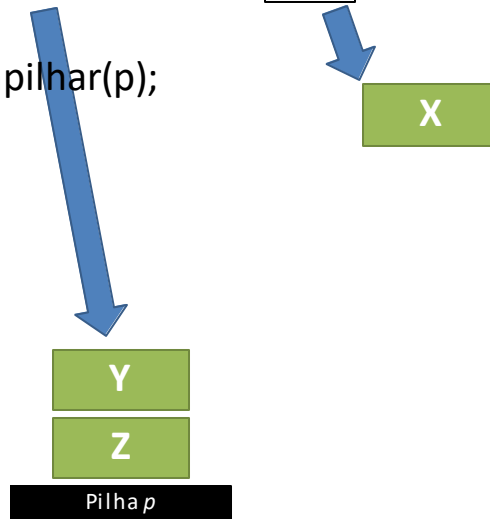
- Pilha Vazia: **Topo**
- Retirar o elemento Y?
 - Não.



Pilhas

- Pilha Vazia: **Topo**

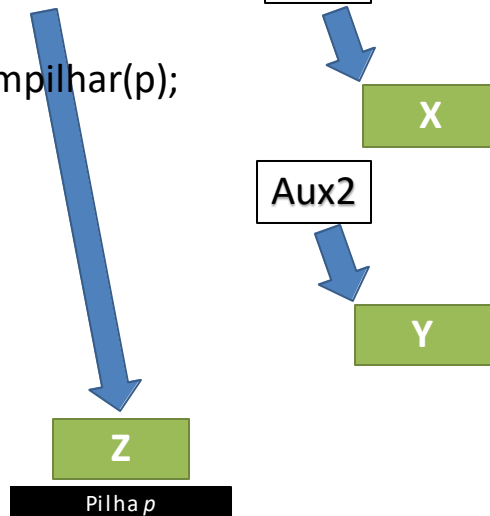
- Aux = Desempilhar(p);



Pilhas

- Pilha Vazia: **Topo**

- Aux2 = Desempilhar(p);



Pilhas

- As pilhas podem ser implementadas em listas encadeadas ou em vetores;
- Vetores:
 - Ter uma variável indicando a posição do topo da pilha;
- Lista Encadeada:
 - Devemos ter uma referência ao elemento do topo da pilha.

Collections

- **Pilha:** ([documentação](#)) ([IME-USP](#))
 - Construir:
 - `Stack()`
 - Adicionar:
 - `push(item)`
 - Remover:
 - `pop()`
 - Retorna o item no topo:
 - `peek()`
 - Testar:
 - `isEmpty()`
 - Tamanho:
 - `size()`

```
class Stack:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def peek(self):
        return self.items[len(self.items)-1]

    def size(self):
        return len(self.items)
```

```
s=Stack()

print(s.isEmpty())
s.push(4)
s.push('dog')
print(s.peek())
s.push(True)
print(s.size())
print(s.isEmpty())
s.push(8.4)
print(s.pop())
print(s.pop())
print(s.size())
```

Collection Stack (pilhas)

- Exercício:
 - Crie um programa que gerencie uma PILHA de TAREFAS a serem cumpridas. As tarefas são Strings que descrevem uma ação a ser executada.
 - O usuário deverá ter duas opções:
 - Inserir tarefa na pilha; e
 - Obter a próxima tarefa da pilha.

Collection Queue (Fila)

- Exercício:
 - Implemente um programa que contemple uma fila de contatos para um *call center*;
 - As opções do programa devem ser:
 - Inserir Contato:
 - Deve solicitar ao usuário os dados e incluir o contato na fila;
 - Próximo Contato:
 - Deverá pegar o Contato do Início da Fila, removê-lo e mostrar os seus dados na tela para o usuário efetuar o contato com o cliente.
 - Sair.